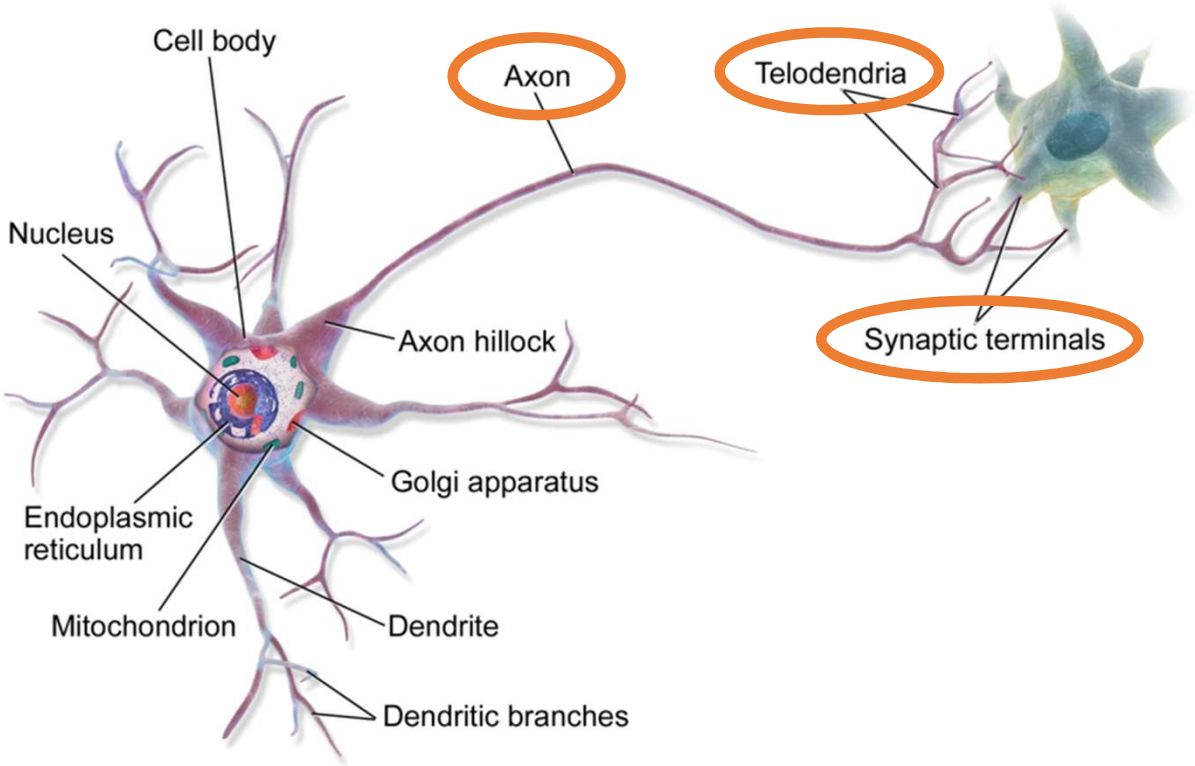
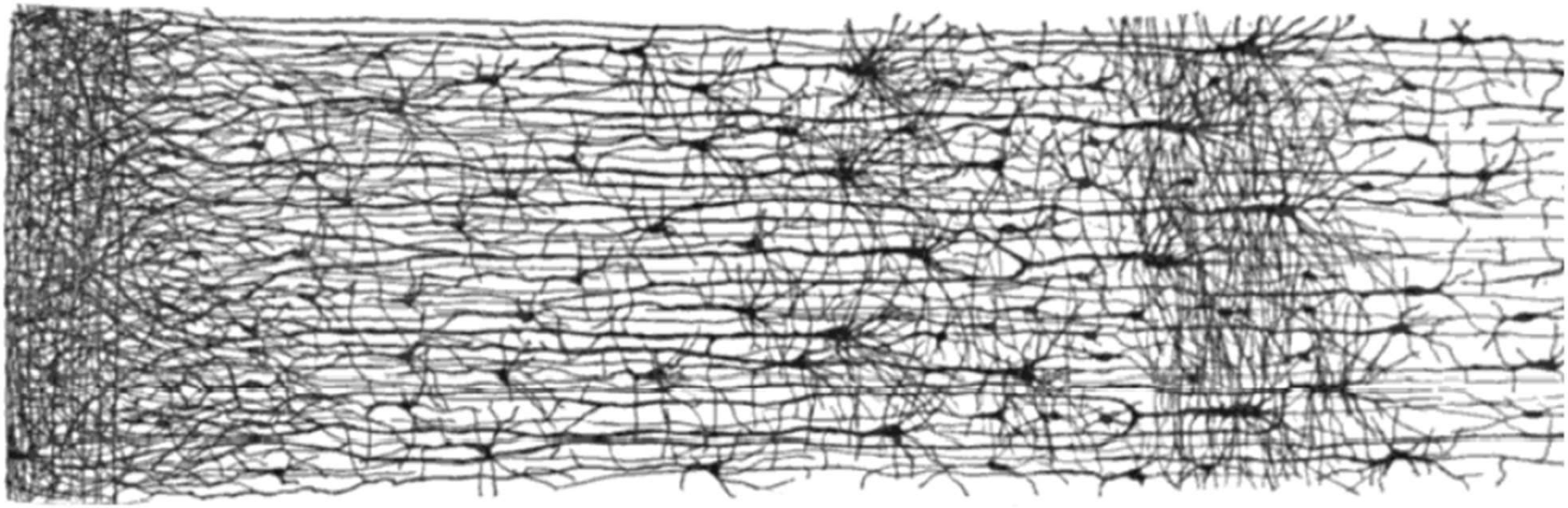
Artificial Neural Networks (ANNs)

- An ML and AI class of algorithms that mimics neural networks of a brain
- Can solve large variety of problems
  - From linear regressions and binary classifications to machine vision, synthetic data generation, cognitive problems, etc.
- Efficient on large volumes of data

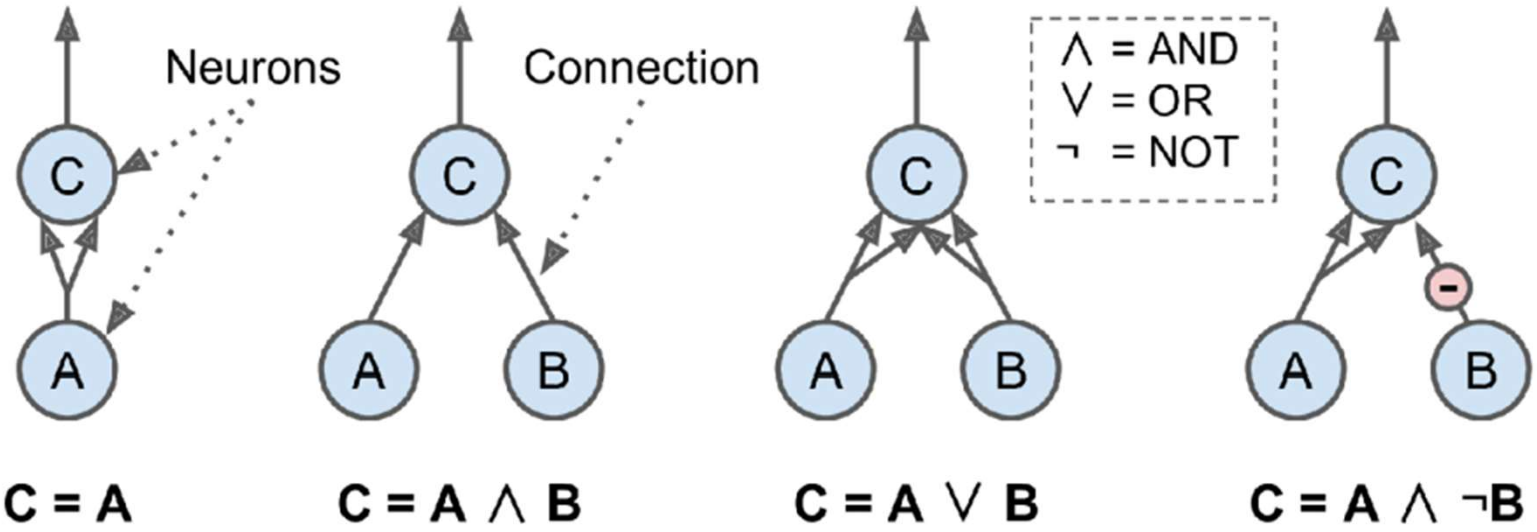
Biological Neurons



Biological Neural Networks (BNNs)



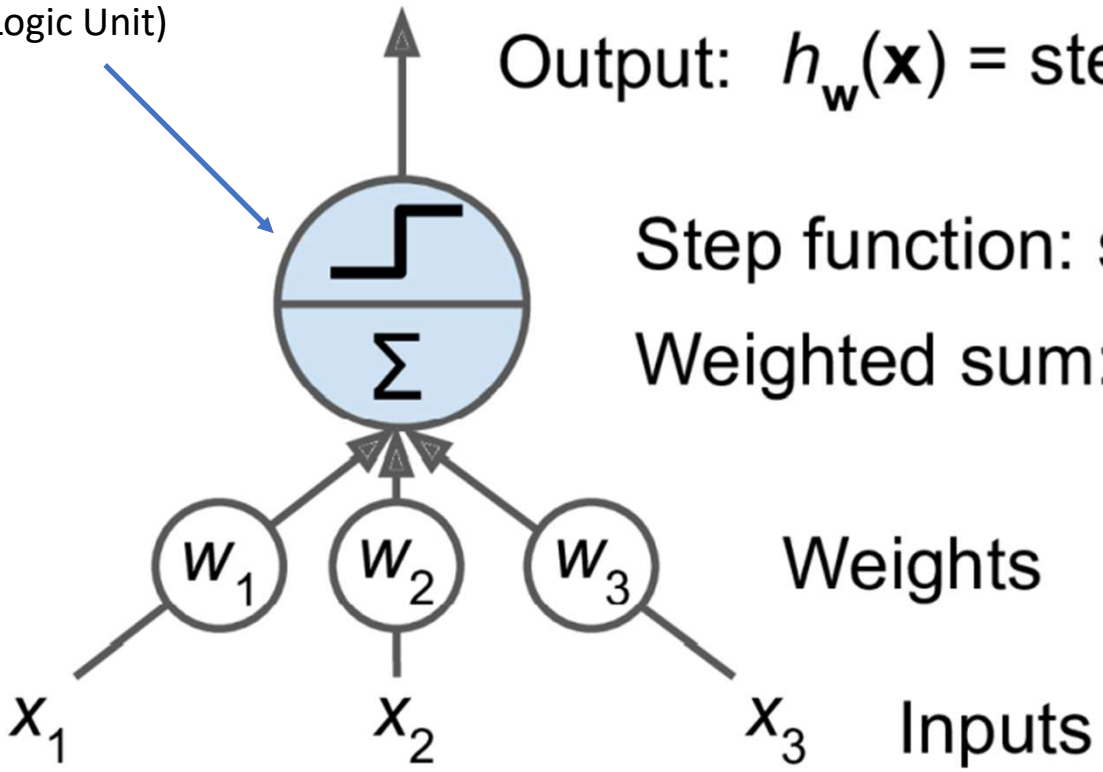
Logical Computations Neurons



The Perceptron

Invented by Frank Rosenblatt in 1957

TLU (Threshold Logic Unit)



Output: $h_{\mathbf{w}}(\mathbf{x}) = \text{step}(\mathbf{x}^T \mathbf{w})$

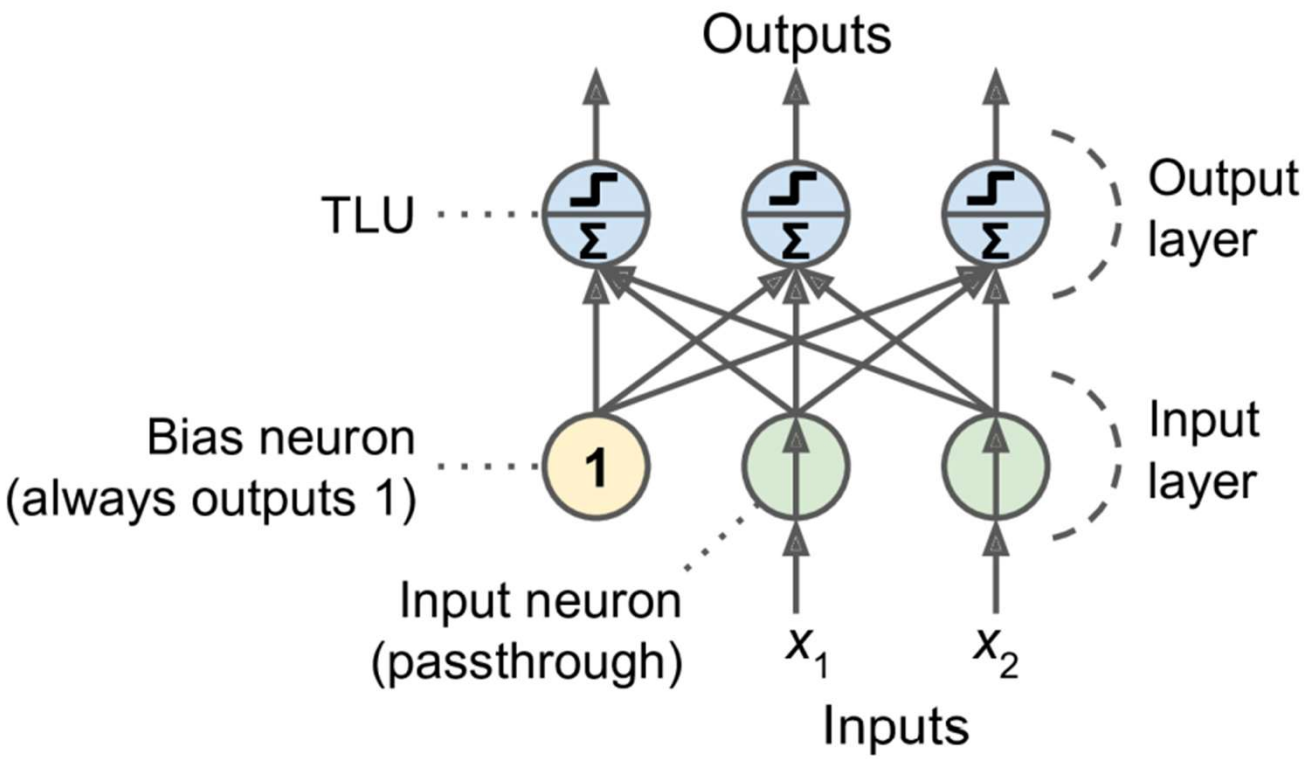
Step function: $\text{step}(z)$

Weighted sum: $z = \mathbf{x}^T \mathbf{w}$

Weights

Inputs

The Perceptron



$$h_{\mathbf{W},\mathbf{b}}(\mathbf{X}) = \phi(\mathbf{X}\mathbf{W} + \mathbf{b})$$

Training A Perceptron

- Donald Hebb in 1949:
  - when a biological neuron triggers another neuron often, the connection between these two neurons grows stronger
- Siegrid Löwel
  - “Cells that fire together, wire together”
- The Perceptron learning rule reinforces connections that help reduce the error

Equation 10-3. Perceptron learning rule (weight update)

$$w_{i,j}^{(\text{next step})} = w_{i,j} + \eta \left(y_j - \hat{y}_j \right) x_i$$

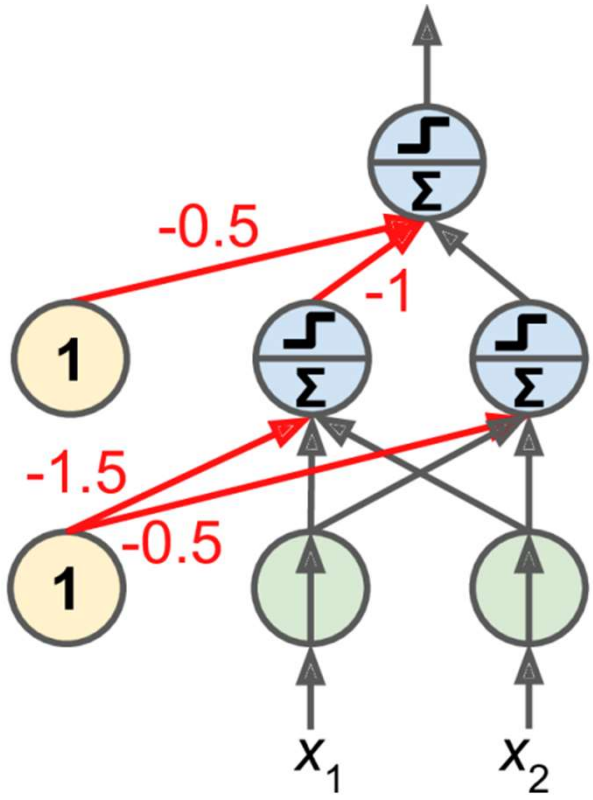
Perceptron Limitations

- The decision boundary of each output neuron is linear:
  - Perceptrons are incapable of learing complex patterns

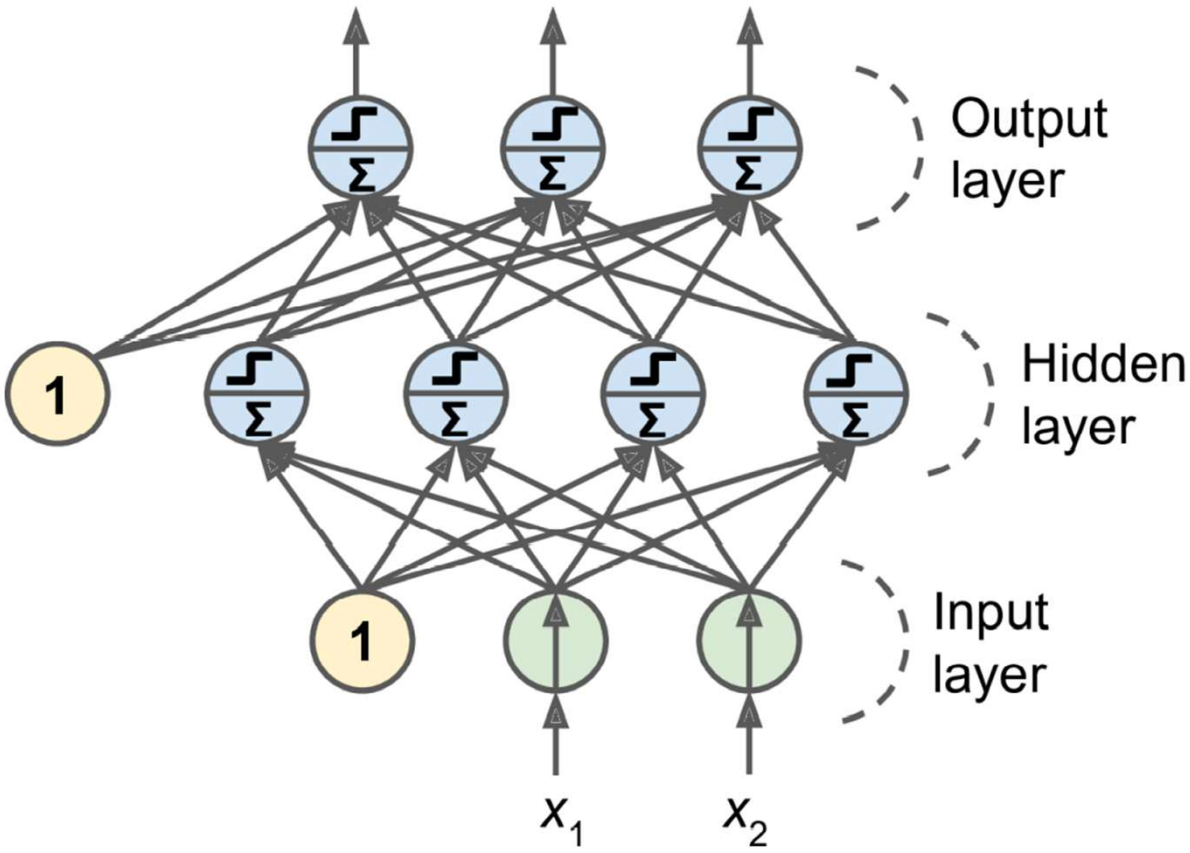
- But if the training instances are linearly separable, the training will converge to a solution

Multilayer Perceptrons

- Single layer Perceptrons cannot solve some trivial problems (e.g. the exclusive OR (XOR) classification)
- Stacking multiple Perceptrons provides a solution



Multilayer Perceptrons



Deep Neural Network (DNN)

ANN that contains a deep stack of hidden layers

Multilayer Perceptrons - Backpropagation

- David Rumelhart, Geoffrey Hinton, and Ronald Williams in 1986
  - Break training data into minibatches
  - **Forward pass**: Each minibatch is passed through the network while saving the intermediate results on each layer
  - Output error is measured (using a loss function)
  - Each output is evaluated by its contribution to the error
  - **Backward pass**: Each layer node contribution to the error is evaluated
  - A Gradient Descent step is used to tweak all the weights in the network

Activation Functions

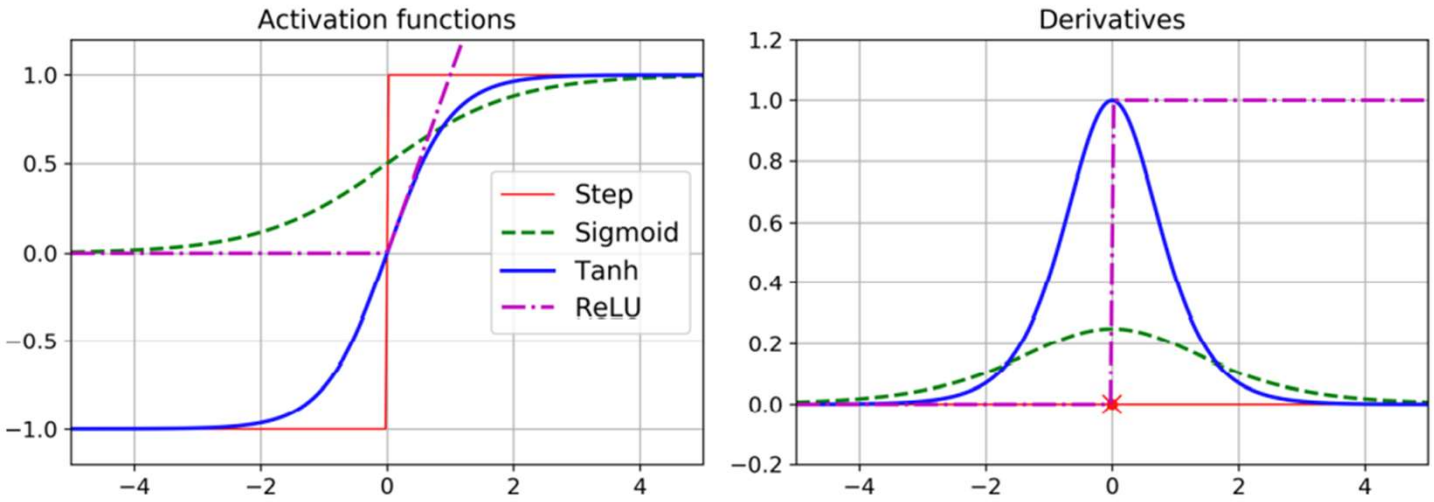
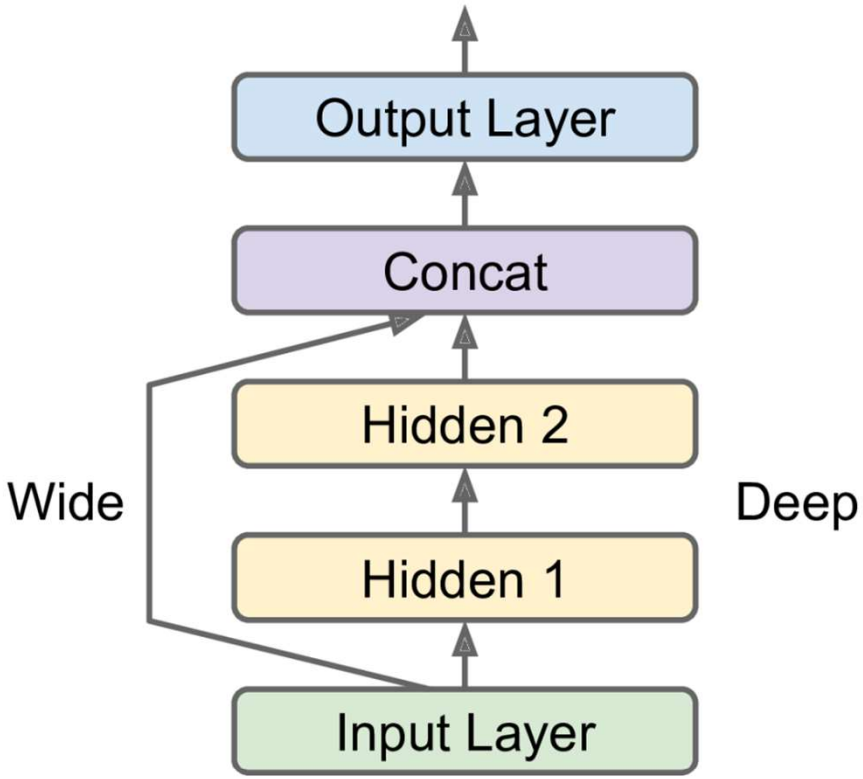


Figure 10-8. Activation functions and their derivatives

Nonlinear activation functions allow DNNs learn complex problems

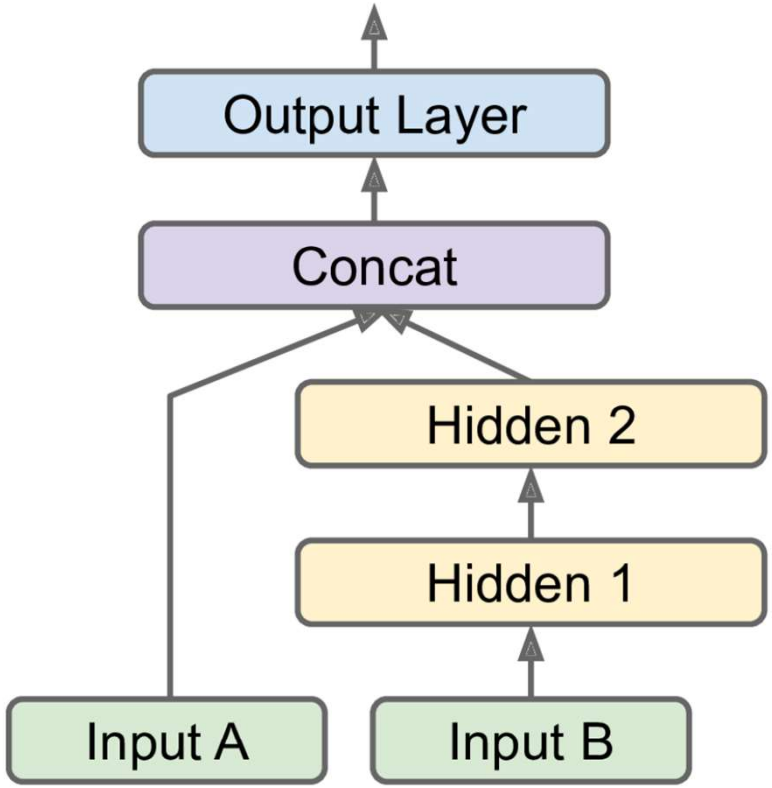
Wide And Deep ANNs



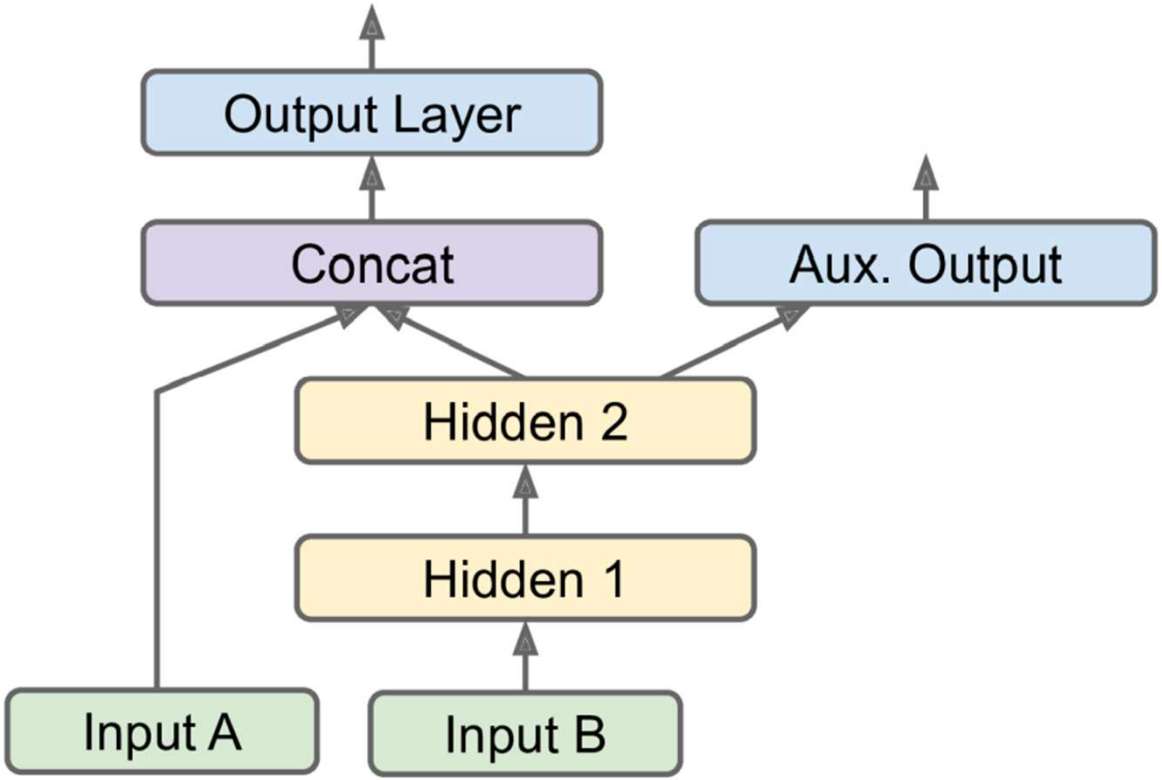
The deep path learns deep patterns

The wide path learns simple patterns

Handling Multiple Inputs



Handling Multiple Outputs



Tuning ANNs

- ANNs have many hyperparameters
  - Number of hidden layers
  - Number of neurons per hidden layer
  - Learning rate
  - Batch size
  - Activation functions
  - Weight initialization
  - …

Tuning ANNs

- Libraries for optimizing hyperparameters
  - Hyperopt
  - Hyperas, kopt, Talos
  - Keras Tuner
  - Scikit-Optimize
  - ...